

System Design Document

Source: docs/SYSTEM_DESIGN.md • Status: Verified & Synchronized

System Design Document

Product: Cubyntra
Organization: Necookie Labs
Document Version: 1.0
Status: V1 Release
Last Updated: 2026-09-26

1. System Overview & Component Responsibilities

The system is decomposed into five primary subsystems coordinated by a centralized Zustand store:

Subsystem	Key Files	Primary Responsibilities
Vision & Scanner	src/vision/camera.ts src/vision/sampling.ts src/vision/color.ts src/vision/stability.ts	MediaStream capture, centered ROI framing, 3x3 pixel aggregation with outlier rejection, HSV/CIELAB conversion, and multi-frame consensus buffering.
Cube Domain	src/cube/types.ts src/cube/constants.ts src/cube/transforms.ts src/cube/validator.ts	Representation of the 54 facelets and 27 cubie coordinates, face rotations, move parsing/inversion, and parity validation.
Solver Subsystem	src/solver/types.ts src/solver/solver.ts	Kociemba two-phase table lookups, move optimization, and simulation verification.
3D Digital Twin	src/three/cubie.ts src/three/arrows.ts src/three/engine.ts src/components/cube/CubeVisualizer.tsx	WebGL scene lifecycle, lighting, camera controls, layer rotation animations, orthogonal matrix snapping, and 3D directional arrows.
State & Presentation	src/stores/useCubyntraStore.ts src/components/	Application state machine, responsive dual-pane layout, move ticker, speed controls, and CV debugger telemetry.

2. End-to-End Sequence Diagram

```
sequenceDiagram
    autonumber
    actor User
    participant Camera as WebRTC Camera
    participant Scanner as CameraScanner
    participant Vision as Color & Stability Engine
    participant Store as useCubyntraStore
    participant Validator as CubeValidator
    participant Solver as KociembaSolver
    participant Three as Three.js Engine

    User->>Store: Click "Scan Physical Cube"
    Store->>Scanner: Mount and initialize camera
    Scanner->>Camera: getUserMedia({ facingMode: 'environment' })
    Camera-->>Scanner: MediaStream attached to <video>
```

```

loop Every Animation Frame
  Scanner-->>Vision: sampleGridFromContext(ctx, roi)
  Vision-->>Scanner: 9x StickerSample[] + confidence
  Scanner-->>Vision: processFrame(samples, face)
  Vision-->>Scanner: stabilityProgress, isStable
  Scanner-->>Store: setClassification(stabilityResult)
end

User-->>Scanner: Tap "Capture Face" (or auto-lock triggers)
Scanner-->>Store: captureFace(ScannedFace)
Note over Store: Repeats for U, F, R, B, L, D (6 Faces)

Store-->>Validator: validateCubeState(reconstructedState)
alt State is Invalid (e.g. Parity Error)
  Validator-->>Store: ValidationResult (issues[])
  Store-->>User: Display ErrorRecoveryModal with Rescan recommendation
else State is Valid
  Validator-->>Store: ValidationResult (valid: true)
  Store-->>Solver: solveCube(cubeState)
  Solver-->>Store: SolveResult (moves[], durationMs)
  Store-->>Three: syncWithCubeState(scrambledState)
  Store-->>User: Display Solution Ready & Move Ticker
end

User-->>Store: Click "Play" or "Step Next"
Store-->>Three: animateMove(move, targetState)
Three-->>Three: Rotate pivot group + Snap orthogonal integers
Three-->>User: 60 FPS animated turn with 3D Directional Arrow

```

3. Scanning State Machine

The scanning lifecycle transitions through deterministic states managed by Zustand:

```

stateDiagram-v2
  [*] --> READY: App Initialization
  READY --> SCANNING: Start Scanning
  READY --> PROCESSING: Load Scramble Demo

  state SCANNING {
    [*] --> SAMPLING: Frame Acquisition
    SAMPLING --> STABILIZING: Consecutive Matches
    STABILIZING --> SAMPLING: Motion Blur / Inconsistency
    STABILIZING --> STABLE_LOCKED: 8 Frames Matching (Progress = 100%)
    STABLE_LOCKED --> FACE_CAPTURED: Capture Face
    FACE_CAPTURED --> ADVANCE_FACE: Next Step < 6
    ADVANCE_FACE --> SAMPLING
  }

  SCANNING --> PROCESSING: All 6 Faces Captured
  PROCESSING --> SOLUTION_READY: Validation & Solving Succeeded
  PROCESSING --> ERROR: Validation Failed (Parity / Count Error)

  ERROR --> SCANNING: Rescan Face / Restart

  state SOLUTION_READY {
    [*] --> IDLE_STEP
    IDLE_STEP --> ANIMATING_MOVE: Step Next / Play
    ANIMATING_MOVE --> IDLE_STEP: Move Snapped (Pause)
  }

  SOLUTION_READY --> SOLVING: Play Active
  SOLVING --> SOLVED: Final Move Executed
  SOLVED --> READY: Reset All

```

4. Domain Data Models & Interfaces

4.1 Cube Domain Model (src/cube/types.ts)

```
export type CubeColor = 'white' | 'yellow' | 'green' | 'blue' | 'red' | 'orange';
export type Face = 'U' | 'R' | 'F' | 'D' | 'L' | 'B';
export type QuarterTurns = 1 | -1 | 2;

export interface CubeMove {
  face: Face;
  quarterTurns: QuarterTurns;
  notation: string;
  description: string;
}

export type FaceStickers = [
  CubeColor, CubeColor, CubeColor,
  CubeColor, CubeColor, CubeColor,
  CubeColor, CubeColor, CubeColor
];

export interface CubeState {
  U: FaceStickers;
  R: FaceStickers;
  F: FaceStickers;
  D: FaceStickers;
  L: FaceStickers;
  B: FaceStickers;
}
```

4.2 Computer Vision Models (src/vision/types.ts)

```
export interface RGBColor { r: number; g: number; b: number; }
export interface HSVColor { h: number; s: number; v: number; }
export interface LABColor { l: number; a: number; b: number; }

export interface StickerSample {
  row: number;
  col: number;
  index: number;
  rgb: RGBColor;
  hsv: HSVColor;
  lab: LABColor;
  predictedColor: CubeColor;
  confidence: number;
  colorScores: Record<CubeColor, number>;
}

export interface FrameClassificationResult {
  stickers: StickerSample[];
  averageConfidence: number;
  isStable: boolean;
  stabilityProgress: number;
  stableFramesCount: number;
  expectedFace: Face;
}
```

5. Concurrency, Performance & Memory Management

5.1 Non-Blocking Sampling Loop

Video frames are extracted using `ctx.getImageData()` within a `requestAnimationFrame` loop. To keep CPU utilization under 15% on mobile devices:

- Only the 9 central sampling patches are extracted, rather than copying the entire video frame buffer.
- React state updates are throttled: store updates occur only on changes in classifications or stability progression.

5.2 WebGL Resource Disposal

When navigating away from the visualizer or unmounting:

- `cancelAnimationFrame` terminates all render loops.
- `mesh.geometry.dispose()` frees buffer geometries.
- `material.dispose()` frees shader programs and textures.
- `renderer.dispose()` releases the WebGL 2.0 rendering context.
- `track.stop()` releases hardware camera lenses.

